Basic Networking Skills for Digital Forensics

Idriss Muhammad Abdullahi
2025/FWSD/11475

Fellowship in Web Application Security & Digital Forensics

26/10/2026

Introduction

PART A

Tools and Workspace

```
┌──(root㉿kali)-[~]
└─# sudo apt update
sudo apt install -y sleuthkit testdisk wireshark tshark libimage-exiftool-perl binwalk steghide unzip p7zip-full john
```

```
┌──(root㉿kali)-[~]
└─# mkdir -p ~/SBT-DF-Case4/{original,work,output,shots,recovered,network}
cd ~/SBT-DF-Case4
```

Tools versions

```
┌──(root㉿kali)-[~/SBT-DF-Case4]
└─# mmls -V; fls -V; photorec --version 2>&1 | head -2; tshark -v | head -1; exiftool -ver; steghide --version
The Sleuth Kit ver 4.14.0
The Sleuth Kit ver 4.14.0
PhotoRec 7.2, Data Recovery Utility, February 2024
Christophe GRENIER <grenier@cgsecurity.org>
Running as user "root" and group "root". This could be dangerous.
Warning: program compiled against libxml 215 using older 214
TShark (Wireshark) 4.6.6.
13.55
steghide version 0.6.0
```

Figure 2: tools and the folder structure of the entire analysis with tools versions for proper investigation

1. Preserving Evidence (Question 1)

```
┌──(root㉿kali)-[~/SBT-DF-Case4]
└─# for f in original/*; do stat -c '%n  %s bytes' "$f"; sha256sum "$f"; done | tee output/E01_original_hashes.txt
original/basic.log  1812 bytes
7918be717c9c820da1921eea6dfb24e461aff471740b90e062faabc2a3494b5f  original/basic.log
original/building.jpg  68661 bytes
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeeef97afdb  original/building.jpg
original/crackWEP40_GPU_Improved_v1.py  3150 bytes
394ec333dcdf72bb1b2f7ab5c22a0ec3232388959463ebf1461ec6c065a462ea  original/crackWEP40_GPU_Improved_v1.py
original/crackWEP40_GPU.py  2807 bytes
c35d3e51b3c974573803476446c708d63e0350668e4e8e87f59f7978e281b909  original/crackWEP40_GPU.py
original/crackWEP40_noGPU_Improved_v1.py  1821 bytes
2d1fa34842c0dd6dc395f77164a39e2d5db21bfe46354b3110f610f307481f54  original/crackWEP40_noGPU_Improved_v1.py
original/crackWEP40_noGPU.py  2573 bytes
9de57a1f9affffb8c48d0f9192246fe6727b4f058c771b48a886312c790acd1c  original/crackWEP40_noGPU.py
original/crackWEP40.py  2573 bytes
9de57a1f9affffb8c48d0f9192246fe6727b4f058c771b48a886312c790acd1c  original/crackWEP40.py
original/DFRWS2005-RODEO.zip  3614418 bytes
e83459a800baed6a3c37551d413ab59fe8dac54c879ed5408001cfed171a0aaa  original/DFRWS2005-RODEO.zip
original/f0335017_She_died_in_February_at_the_age_of_74.doc  35840 bytes
760d78c3a9725a5832c886cbe8986804891a7fd71a5fd16ff191f447b0157bcc  original/f0335017_She_died_in_February_at_the_age_of_74.doc
original/file.xz  12837660 bytes
dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6  original/file.xz
original/findseed.py  735 bytes
1ac9d1641a4717bcfb5ee291ae08a13d2ebcffed78045f7d354f09dd6275915e  original/findseed.py
original/image2.log  73692 bytes
4d9bc50bc05213828ff7807f8bd6bb2b40f5f8da127c20936b774086187dd594  original/image2.log
original/image.html  151 bytes
e6fc538c7fa723b5c73947fcdf75446da53a24713a9cbcaeb0d1a3ea1936fb3d  original/image.html
original/image.log  1848 bytes
d87437bb3555f93ecbea0b10cf19661da1210f23138b27bb1d33729b7c469416  original/image.log
original/lab_files  4096 bytes
```

Figure 2: (E01) all original evidence files well preserved and hashed (SHA256Sum)

File	Size	Hashes

basic.log	1812 bytes	7918be717c9c820da1921eea6dfb24e461aff471740b90e062faabc2a3494b5f
building.jpg	68661 bytes	a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeeef97afdb
crackWEP40_GPU_Improved_v1.py	3150 bytes	394ec333dcdf72bb1b2f7ab5c22a0ec3232388959463ebf1461ec6c065a462ea
DFRWS2005-RODEO.zip	3614418 bytes	e83459a800baed6a3c37551d413ab59fe8dac54c879ed5408001cfed171a0aaa
f0335017_She_died_in_February_at_the_age_of_74.doc	35840 bytes	760d78c3a9725a5832c886cbe8986804891a7fd71a5fd16ff191f447b0157bcc
file.xz	12837660 bytes	dd54144caef34f228bfb4b87a9101ca7173969376f7ed357bd068061d4f4b8d6
image2.log	73692 bytes	4d9bc50bc05213828ff7807f8bd6bb2b40f5f8da127c20936b774086187dd594
image.html	151 bytes	e6fc538c7fa723b5c73947fcdf75446da53a24713a9cbcaeb0d1a3ea1936fb3d
mySYNFloodCapture.pcap	260716 bytes	14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5
rhino2.log	292604 bytes	41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde
rhino3.log	226094 bytes	7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46
rhino.log	3187907 bytes	64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2
smtp.pcap	27850 bytes	17ad230db1b6fd5dd18eb311092df1cf6eb162054bdb47697b89bef5a86a47ab
synflood.py	559 bytes	31478ed7cd25f0ac046480e1b1656aed6f7851424feca3e1c2bf715735d75800
stegdetect.7z	3619583 bytes	d58693b0bbc9dd29f05785ef28cb386ede0706fe5d6f6a40daa6c9625b79945c
RHINOUSB.dd	253424 KB	ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65

2. Examining the disk image before recovery (Question 2)
   Using mmls work/RHINOUSB.dd | tee output/E02_mmls.txt

fsstat work/RHINOUSB.dd | tee output/E02_fsstat.txt
fls -r -p work/RHINOUSB.dd | tee output/E02_fls_allocated.txt
fls -r -d -p work/RHINOUSB.dd | tee output/E02_fls_deleted.txt

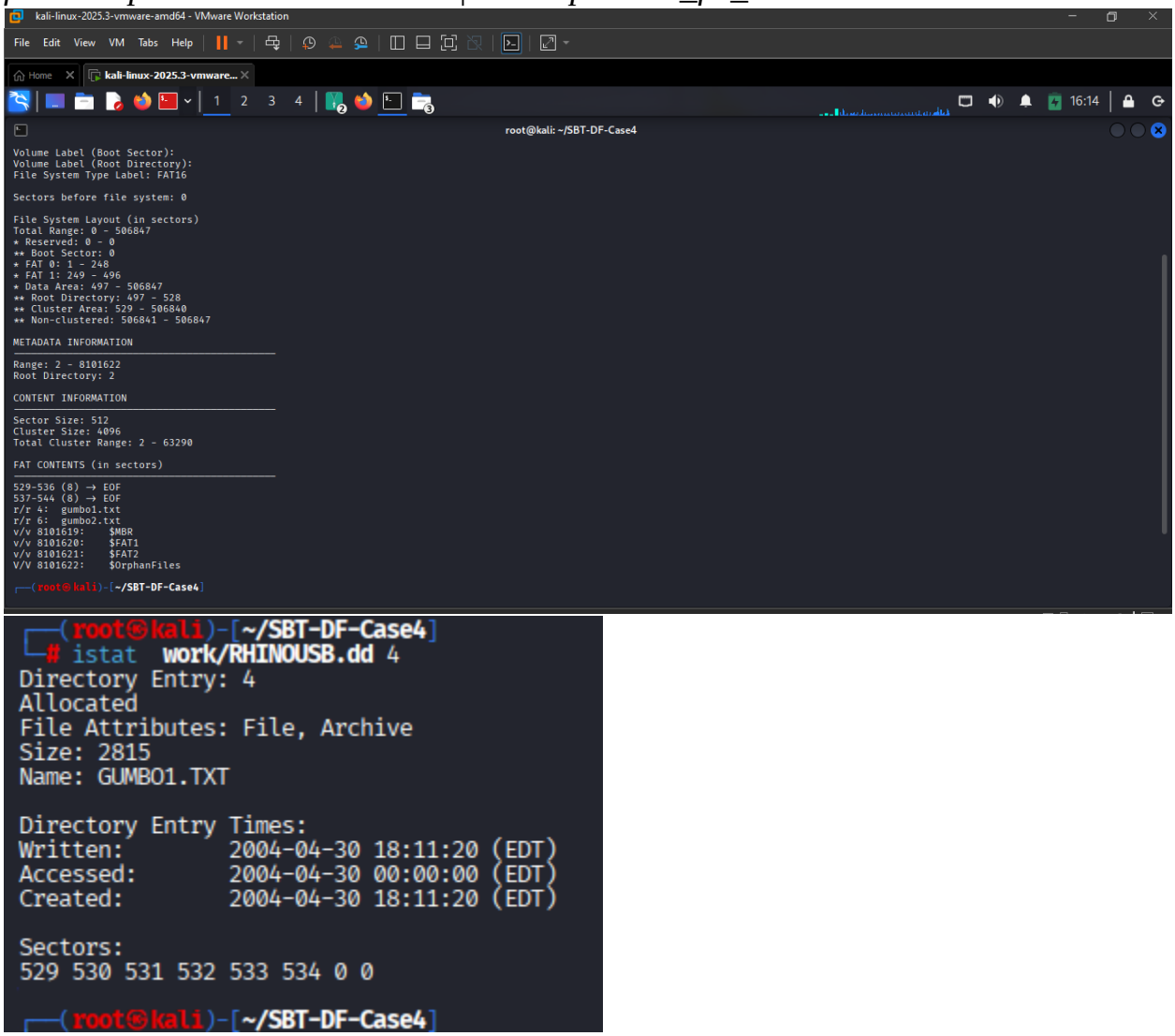


Figure 3. E02 the screenshots above shows the examination of the disk image before (file system details, allocated files and deleted entries only) recovery.

File system: FAT16, no volume label, 0 sectors before the file system
Total Sectores:) - 506,847 (512 B/sector, 4,096 B/cluster, clusters 2 – 63,290)
Layout: Boot sector 0 | FAT0 1–248 | FAT1 249–496 | Root dir 497–528 | Cluster area 529–506,840
Files found
a. gumbo1.txt (r/r 4), sectors 529–536, single cluster, allocated

b. gumbo2.txt (r/r 6), sectors 537–544, single cluster, allocated
Virtual entries: $MBR, $FAT1, $FAT2, $OrphanFiles (8101619–8101622)
Observation:

3. Recovering deleted files (Question 3)

Using PhotoRec (a free and open-source utility software for data recovery)



Figure 4; E03. File recover using Photorec was successfully achieved and 132 files were recovered and saved in the recovery folder with the directory name (recup_dir.1).

Navigating to the recovery folder to confirm the recovered files

```
┌──(root㉿kali)-[~/SBT-DF-Case4/recovered]
└─# ls -l
total 4
drwxrwxr-x 2 root root 4096 Oct 25 20:55 recup_dir.1

┌──(root㉿kali)-[~/SBT-DF-Case4/recovered]
└─# cd recup_dir.1

┌──(root㉿kali)-[~/SBT-DF-Case4/recovered/recup_dir.1]
└─# ls
f0000545.txt                                f0336457.txt  f0343657.txt  f0350849.txt  f0358049.txt  f0365249.txt  f0372441.txt  f0379641.txt  f0386833.txt  f0394033.txt
f0104057.jpg                                f0336969.txt  f0344585.txt  f0351369.txt  f0358561.txt  f0365761.txt  f0372953.txt  f0380153.txt  f0387353.txt  f0394545.txt
f0104249.jpg                                f0337489.txt  f0344681.txt  f0351881.txt  f0359073.txt  f0366273.txt  f0373473.txt  f0380665.txt  f0387865.txt  f0395065.txt
f0105065.jpg                                f0338001.txt  f0345201.txt  f0352393.txt  f0359593.txt  f0366785.txt  f0373985.txt  f0381185.txt  f0388377.txt  f0395577.txt
f0105873.jpg                                f0338513.txt  f0345713.txt  f0352905.txt  f0360105.txt  f0367305.txt  f0374497.txt  f0381697.txt  f0388889.txt  f0396089.txt
f0106393.jpg                                f0339025.txt  f0346225.txt  f0353425.txt  f0360969.txt  f0367817.txt  f0375009.txt  f0382209.txt  f0389409.txt  f0396601.txt
f0106409.jpg                                f0339545.txt  f0346737.txt  f0353937.txt  f0361137.txt  f0368329.txt  f0375529.txt  f0382721.txt  f0389921.txt  report.xml
f0106865.gif                                f0340057.txt  f0347257.txt  f0354449.txt  f0361649.txt  f0368841.txt  f0376041.txt  f0383241.txt  f0390433.txt
f0106889.gif                                f0340569.txt  f0347769.txt  f0354961.txt  f0362161.txt  f0369361.txt  f0376553.txt  f0383753.txt  f0390945.txt
f0106905.txt                                f0341081.txt  f0348281.txt  f0355481.txt  f0362673.txt  f0369873.txt  f0377353.txt  f0384265.txt  f0391465.txt
f0335017_She_died_in_February_at_the_age_of_74.doc  f0341601.txt  f0348793.txt  f0355993.txt  f0363193.txt  f0370385.txt  f0377585.txt  f0384777.txt  f0391977.txt
f0335081.jpg                                f0342113.txt  f0349313.txt  f0356505.txt  f0363705.txt  f0370897.txt  f0378097.txt  f0385297.txt  f0392489.txt
f0335601.txt                                f0342625.txt  f0349825.txt  f0357017.txt  f0364217.txt  f0371417.txt  f0378609.txt  f0385809.txt  f0393001.txt
f0335945.txt                                f0343137.txt  f0350337.txt  f0357537.txt  f0364729.txt  f0371929.txt  f0379129.txt  f0386321.txt  f0393737.txt
```

Opening some of the recovered files for display using (Feh)

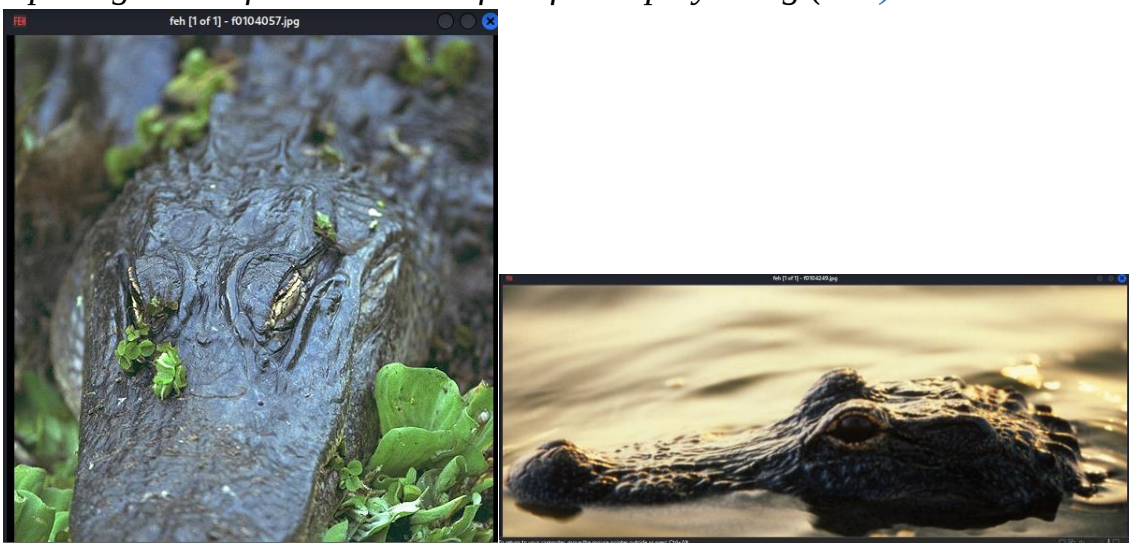


Figure 5; E04. Listed recovered files and some displayed .jpg recovered files.

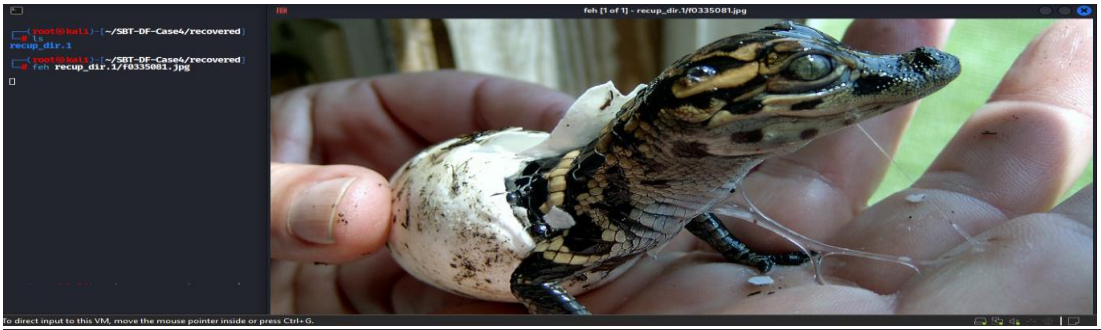
Hashing unique files and duplicates separately because duplicate hashes mean identical files

```
┌──(root㉿kali)-[~/SBT-DF-Case4/recovered]
└─# find recup_dir.1 -type f -exec sha256sum {} \; | sort | tee ~/SBT-DF-Case4/output/E03_recovered_hashes.txt
033f1c0e61713b3bd1609ea86501d1379ae0ebc0ca770c3df7921ef86ad7d6e3  recup_dir.1/f0349313.txt
07acfef050fdf1d548fed79260b319c1968e9c4985c397451c947093ac5cfdbc  recup_dir.1/f0347769.txt
08a0b5e4de6844dec73217af4d7dff3f3ff0486c1548e18673a5c75aa98230c4  recup_dir.1/f0358561.txt
08a277a78b34d9395ca7051ab131862c31e5fb42368ab5ef99544a7b7ef52cf1  recup_dir.1/f0375009.txt
09cd38e4f63c5326fbe3b3e7064711c32f174a76773916c4893089420c95f42f  recup_dir.1/f0355993.txt

┌──(root㉿kali)-[~/SBT-DF-Case4/recovered]
└─# sort ~/SBT-DF-Case4/output/E03_recovered_hashes.txt | awk '{print $1}' | uniq -d
f92654d9ee17ab6b684b09de01cf0bc4076383c007964946d3f31577447596fb

┌──(root㉿kali)-[~/SBT-DF-Case4/recovered]
```

The hashing output shows only one duplicate file which is (f0335081.jpg) was saved in the output folder as evidence.



File output saved in evidence folder as evidence

What happened to the hard drive and USB?



4. Metadata and hidden content (Question 4)
   Reading the metadata and hidden content of this examination was carried out using Exiftool and Binwalk

```
┌──(root㉿kali)-[~/SBT-DF-Case4]
└─# exiftool -a -G1 -s recovered/recup_dir.1/*.jpg recovered/recup_dir.1/*.txt | tee
output/E04_exif_exiftool.txt
binwalk recovered/recup_dir.1/*.jpg recovered/recup_dir.1/*.txt | tee output/E04_binw
alk_binwalk.txt
======== recovered/recup_dir.1/f0104057.jpg
[ExifTool]      ExifToolVersion                 : 13.55
[System]        FileName                        : f0104057.jpg
[System]        Directory                       : recovered/recup_dir.1
[System]        FileSize                        : 96 kB
[System]        FileModifyDate                  : 2026:10:25 20:55:28-04:00
[System]        FileAccessDate                  : 2026:10:25 21:12:01-04:00
[System]        FileInodeChangeDate             : 2026:10:25 20:55:28-04:00
[System]        FilePermissions                 : -rw-rw-r--
[File]          FileType                        : JPEG
[File]          FileTypeExtension               : jpg
[File]          MIMEType                        : image/jpeg
[File]          Comment                         : copyright 2000 philg@mit.edu
[File]          ImageWidth                      : 528
[File]          ImageHeight                     : 792
[File]          EncodingProcess                 : Baseline DCT, Huffman coding
[File]          BitsPerSample                   : 8
[File]          ColorComponents                 : 3
[File]          YCbCrSubSampling                : YCbCr4:4:4 (1 1)
```

Figure 6; E0 5. The flag -a I use above shows duplicate tags, -G1 labels each tag with its group and -s prints short tag names. Full file (.txt) successfully stored for evidence in the output folder for detailed as Exiftool.txt and Binwalk.txt.

5. FTP capture (Question 5)

Opening wireshark for rhino.log file

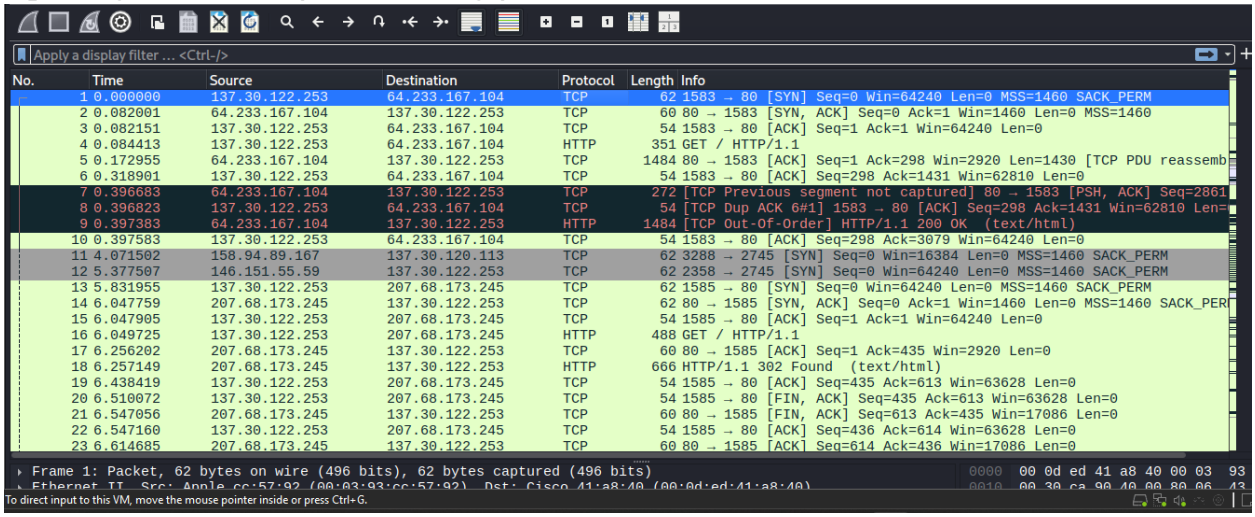
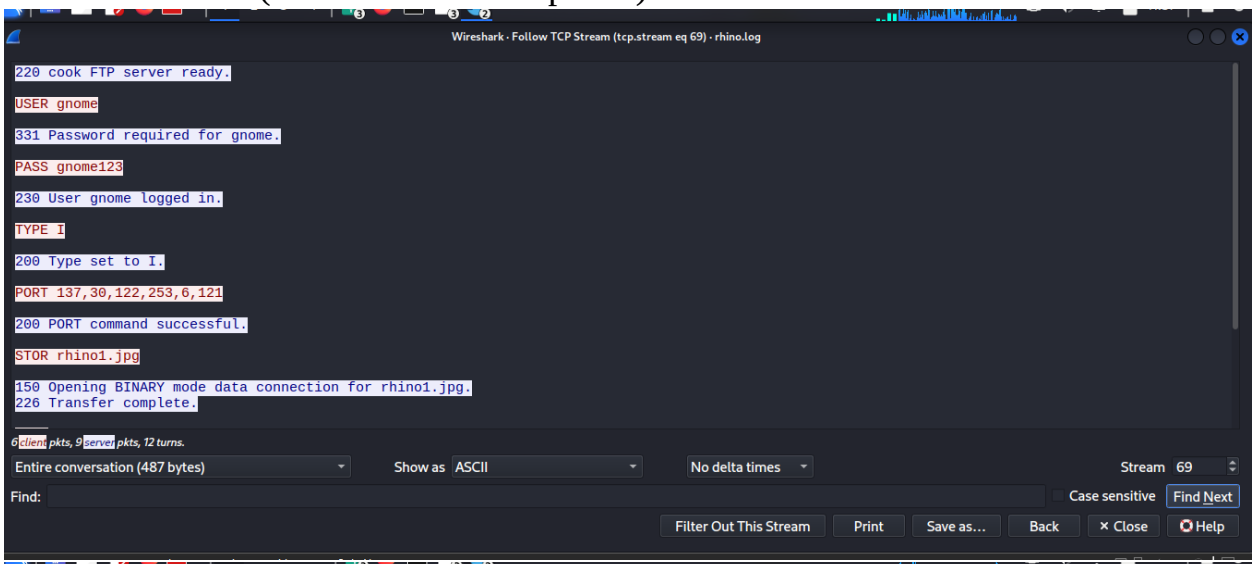
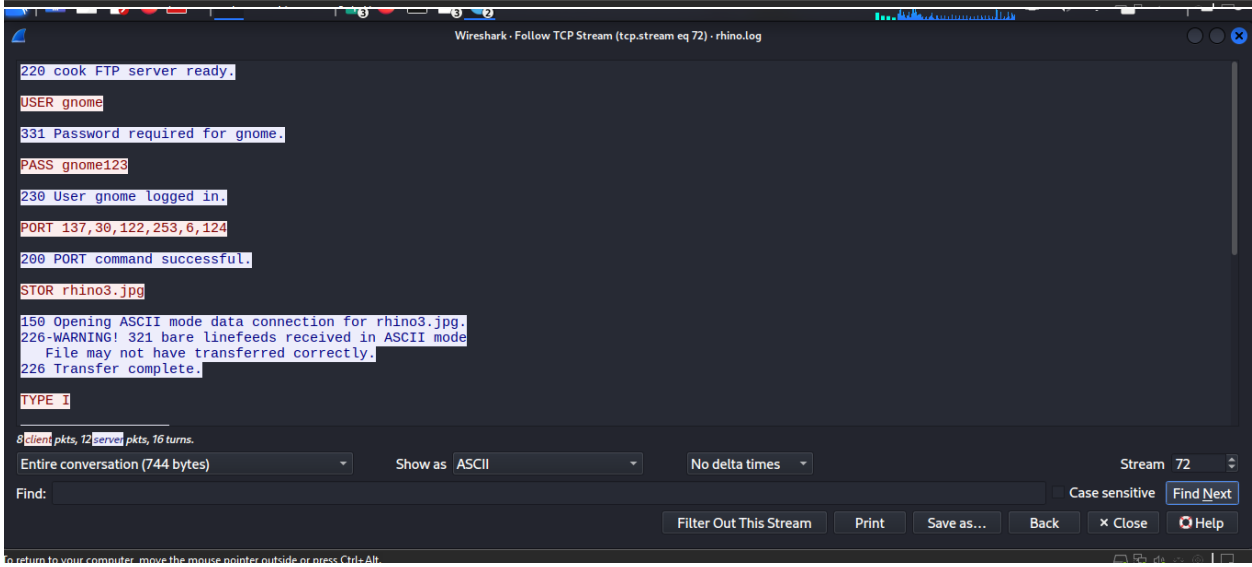


*Figure 6; full rhino.log wireshark triage. All transfers are **uploads from client to server**, because every transfer uses STOR. The client uses **active mode** (PORT), so the server opens the TCP data connection back to the client. The file bytes still flow client → server.*

Following TCP Stream to locate control and data sessions.

1. Control sessions (commands and response)





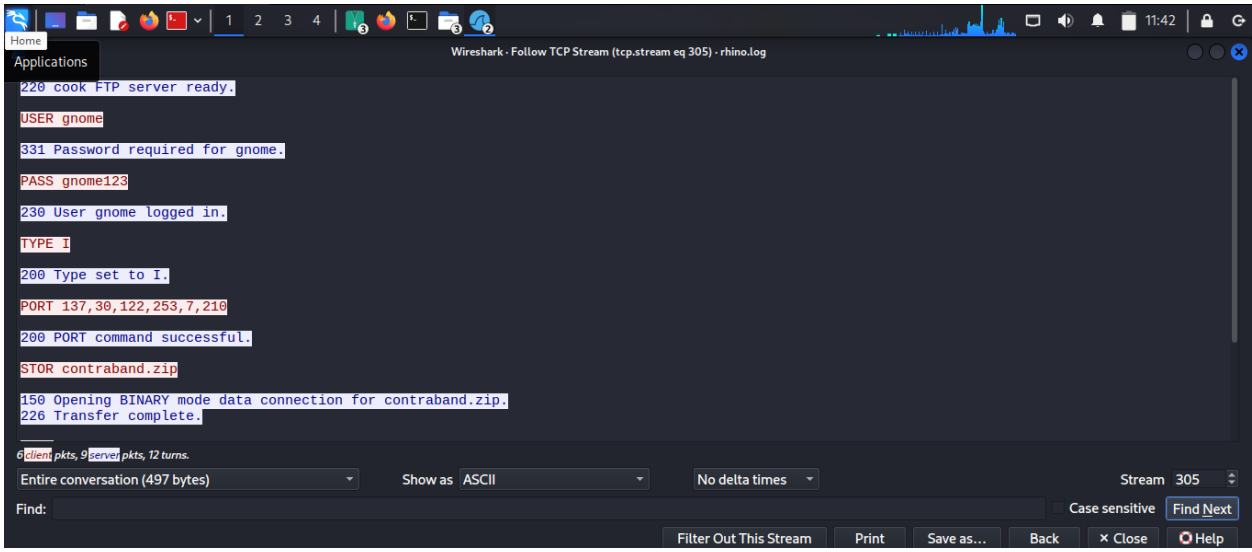


Figure 7; E06. Followed TCP Stream of rhino1.jpg, rhino3.jpg and contraband.zip file to locate control and data sessions

1.2Sensitive evidence: the control stream contains the plaintext login. Treat any visible credential as evidence, redact it in reports, and do not reuse it.

6. HTTP captures (Question 7)

6.1 Rhino2.log (investigating Investigating http traffics contain .jpg, investigating http traffics contain .gif and Investigating email traffics)

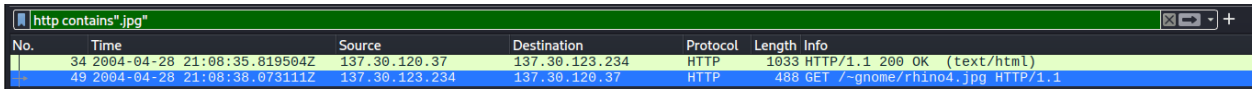


Figure 8. E07. the file rhino2.log was examined with the filtered sequence of http contains ".jpg" (Searching suspect's http requests to download .jpg). The source and the destination ip addresses indicate who is the user and GET is used to request data from the resource.

6.2following who owns the IP

```
┌──(root㉿kali)-[~/SBT-DF-Case4]
└─# whois 137.30.123.234

#
# ARIN WHOIS data and services are subject to the Terms of Use
# available at: https://www.arin.net/resources/registry/whois/tou/
#
# If you see inaccuracies in the results, please report at
# https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
#
# Copyright 1997-2026, American Registry for Internet Numbers, Ltd.
#


NetRange:       137.30.0.0 - 137.30.255.255
CIDR:           137.30.0.0/16
NetName:        UNO-NET-BIG
NetHandle:      NET-137-30-0-0-1
Parent:         NET137 (NET-137-0-0-0-0)
NetType:        Direct Allocation
OriginAS:
Organization:   University of New Orleans (UNO)
RegDate:        1989-10-17
Updated:        2021-12-14
Ref:            https://rdap.arin.net/registry/ip/137.30.0.0


OrgName:        University of New Orleans
OrgId:          UNO
Address:        Computer Research Building
Address:         Lakefront Campus
City:           New Orleans
StateProv:      LA
```

Figure 9. successfully followed who owns the IP which information from name, address and other important data's.

Follow rhino4.jpg HTTP traffic from the above figure 8 (TCP Stream)



Figure 12. followed HTTP Stream (tcp.stream eq 1). The packets from the server to the suspect's machine includes rhino4.jpg and the magic number is GIF89a.....

Timeline

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
1. Media preservation	**2026-10-25 16:14**	**E-01**	**Acquisition notes, hash log**	**Acquisition recorded, write-blocker note or acquisition hash was present.**	**Integrity of the disk image and capture files cannot be asserted until MD5/SHA-256 hashes are recorded for E-01 to E-04. Do this before any other step.**
2. Disk image / file-	**2004-04-30**	**E-02**	**fsstat output, ~/SBT-**	**FAT16; no volume label; 512 B sectors, 4,096 B**	**Standard FAT16 layout with two FAT copies. fsstat**

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
system review	18:11:20		DF-Case4	clusters (clusters 2–63,290). Layout: boot sector 0, FAT0 1–248, FAT1 249–496, root directory 497–528, cluster area 529–506,840, non-clustered 506,841–506,847.	gives structure only, not file times, so this row cannot yet be placed on the time axis.
4. FTP transfer activity: session 1	2004-04-26 22:21:39.998 to 22:21:55.390	E-06	Wireshark, ftp filter, frames 1529–1616	Client 137.30.122.253 logged in to server 137.30.120.40 (account gnome, password redacted; server replied 230). STOR rhino1.jpg in frame 1546 at 22:21:49.954. Server 150 (frame 1550, 22:21:49.991) and 226 Transfer complete (frame 1612, 22:21:50.180). Server reported 65,703 bytes in 1 file (frame 1615).	Upload, client to server, in active mode (PORT frame 1544, data port 1657). Object boundary is the data stream between the 150 and 226 responses. File bytes not yet extracted or validated.
4b. FTP	2004-04-26	E-06	Wireshark, ftp	Same account logged in (frames	Two uploads of the same file name. No

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
transfer activity: session 2	22:21:56.420 to 22:22:22.088		filter, frames 1623–1860	1625–1631). STOR rhino3.jpg in frame 1649 at 22:22:08.906 (PORT frame 1647, data port 1660) and again in frame 1763 at 22:22:16.091. Second transfer: 150 at 22:22:16.116 (frame 1767), 226 at 22:22:16.263 (frame 1851). Server reported 193,797 bytes (frame 1859).	rhino2.jpg seen. Frames 1650–1761 were not visible, so the outcome of the first STOR is unconfirmed (re-upload, overwrite or failure).
4c. FTP transfer activity: session 3	2004-04-26 22:26:35.118 to 22:26:48.548	E-06	Wireshark, ftp filter, frames 5624–5843	Same account logged in (frames 5633–5639). TYPE I frame 5641. STOR contraband.zip in frame 5647 at 22:26:46.672 (PORT frame 5645, data port 2002). 150 at 22:26:46.699 (frame 5651), 226 at 22:26:46.877 (frame 5839). Server reported 230,566 bytes (frame 5842).	Binary upload of a ZIP archive, client to server. Contents are not examined; the file name alone does not establish what it holds.

Stage	UTC timestamp	Evidence ID	Source artefact	Observed fact	Interpretation and limit
5a. HTTP transfer activity: rhino4.jpg	**2004-04-28 21:06:35**	**E-06**	**rhino2.log, HTTP GET and response**	**Suspect requested a .jpg by HTTP GET. Server-to-suspect packets followed to start at JPEG magic FF D8 and end at trailer FF D9. Raw bytes carved, saved as rhino4.jpg, and displayed. MD5 shown in the deck: aa64102afff71b93ed61fb100af8d52a.**	**Download. Hash comes from the deck's worked example; recompute it on your own extraction before citing. Image content and IP ownership (WHOIS) to be recorded.**
5b. HTTP transfer activity: rhino5.gif	**2004-04-28 21:06:35**	**E-06**	**rhino2.log, HTTP, packet 312**	**GIF hosted by cs.uno.edu; image data in packet 312, exported and saved as rhino5.gif. Magic number and trailer verified. MD5 shown in the deck: 1e90b7f70b2ecb605898524a88269029.**	**Download. Five HTTP image responses were listed; only this one was followed, so the other four are unresolved. Recompute the hash on your own export.**

Conclusion

The network evidence shows file-transfer activity involving rhino-named content and one archive named contraband.zip. Between 22:21:39.998 and 22:26:48.548

UTC on 2004-04-26, a client at 137.30.122.253 logged in three times to the FTP server at 137.30.120.40 using the account gnome and uploaded rhino1.jpg, rhino3.jpg (two STOR commands) and contraband.zip, with the server confirming each completed transfer. The HTTP captures show a suspect machine downloading rhino4.jpg and rhino5.gif (hosted by cs.uno.edu), and an executable saved as rhino.exe from the second capture. The images and executable were reconstructed by carving the traffic and checking file signatures (FF D8 / FF D9 for JPEG, the GIF header and trailer, and the executable header).

The disk image does not corroborate possession on its own. It is a FAT16 volume whose only allocated files are gumbo1.txt and gumbo2.txt, with a gap in the entry numbering and an unexamined $OrphanFiles area that may hold deleted material.